

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Minimax Search:** A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning:** A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Turing Test:** An operational test for intelligent behavior where a human interrogator tries to distinguish between a human and a machine based on their text-based responses.
- **Game Tree:** A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing the outcomes.
- **Perceptron:** A fundamental unit in neural networks that takes multiple inputs, applies weights to them, sums the weighted inputs, and produces an output based on a threshold.
- **Reinforcement Learning:** A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.
- **Matrix:** A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform:** A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c .
- **Matrix Multiplication:** An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of the i -th row of the first matrix and the j -th column of the second matrix. The inner dimensions must match.
- **Matrix Inverse:** For a square matrix A , its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition:** An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T .
- **Derivative:** A measure of how a function changes with respect to a change in its input.
- **Partial Derivative:** A derivative of a function of multiple variables with respect to one of those variables, holding the others constant.
- **Sigmoid Function:** A mathematical function having a characteristic "S"-shaped curve, often used as an activation function in neural networks. Its derivative is shown in slide 21.
- **Inductive Reasoning:** The process of deriving general principles from specific observations.
- **Linear Regression:** A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.

- **Model:** A mathematical representation of a system or process, used to make predictions or understand relationships between variables.
- **Overfitting:** A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Loss Function:** A function that quantifies the difference between the predicted and actual values in a machine learning model.
- **Decision Boundary:** A line or hyperplane that separates different classes of data points in a classification problem.
- **Stochastic Gradient Descent:** An iterative optimization algorithm that updates model parameters using the gradient calculated from a randomly selected subset of the training data.
- **Logistic Regression:** A statistical model that uses a logistic function to model the probability of a binary outcome.
- **Perceptron Algorithm:** An iterative algorithm used for binary classification that updates the weights of a linear model based on misclassified data points.
- **Empirical Risk Minimizer:** A learning principle that aims to find the model that minimizes the average loss over the training data.
- **Binary Classification:** A type of classification problem where the goal is to assign data points to one of two classes.
- **Gradient Descent:** An iterative optimization algorithm that updates model parameters by moving in the direction of the negative gradient of the loss function.
- **\textbf{Image Classification}:** The task of assigning a predefined category label to an input image.
- **\textbf{Semantic Gap}:** The difference between the human interpretation of an image and its low-level numerical representation by a computer.
- **\textbf{Viewpoint Variation}:** The changes in an object's appearance due to changes in the viewing angle.
- **\textbf{Illumination}:** The lighting conditions affecting an image, causing variations in brightness and contrast.
- **\textbf{Deformation}:** Changes in an object's shape or pose.
- **\textbf{Occlusion}:** The partial or complete obstruction of an object by another object or element in an image.
- **\textbf{Background Clutter}:** Distracting elements in the background of an image that interfere with object recognition.
- **\textbf{Invariant Features}:** Features of an object that remain consistent across different variations in viewpoint, lighting, or pose.
- **\textbf{Bag of Features}:** A representation of an image as an unordered collection of local features (visual words).
- **\textbf{Visual Words}:** Basic visual elements extracted from an image, representing its content in a way that is less sensitive to variations.
- **\textbf{Category Models}:** Statistical representations of the frequency of visual words for different object categories.

- **\textbf{Feature Vector $\mathbf{f}$ }**: A numerical representation of an object's features.
- **\textbf{Orderless Document Representation}**: Representing a document by the frequencies of words from a dictionary, ignoring word order.
- **\textbf{Robustness}**: The ability of extracted features to remain consistent despite noise, detection errors, illumination changes, occlusion, and shape distortions.
- **\textbf{Invariance}**: The ability of extracted features to remain consistent despite geometric transformations (translation, rotation, scale).
- **\textbf{Visual Dictionary}**: A collection of visual words used to represent and categorize images.
- **\textbf{Textons}**: Basic elements or patterns that characterize a texture.
- **\textbf{Nearest Neighbor (NN) Classifier}**: A classification algorithm that assigns the label of the nearest training data point to a test data point.
- **\textbf{Noise}**: Random variations or unwanted disturbances in an image.
- **\textbf{Salt and Pepper Noise}**: Random occurrences of black and white pixels in an image.
- **\textbf{Impulse Noise}**: Random occurrences of white pixels in an image.
- **\textbf{Gaussian Noise}**: Variations in intensity drawn from a Gaussian normal distribution.
- **\textbf{Moving Average}**: A method for noise reduction that replaces each pixel with the average of its neighboring pixels.
- **\textbf{Weighted Moving Average}**: A moving average where neighboring pixels are weighted differently.
- **\textbf{Image Filtering}**: A process of computing a function of the local neighborhood at each pixel in an image.
- **\textbf{Filter/Mask}**: A matrix that specifies how to combine values from neighboring pixels in image filtering.
- **\textbf{Blur Filter}**: A filter that smooths an image by averaging pixel values in a local neighborhood.
- **\textbf{Identity Filter}**: A filter that leaves the image unchanged.
- **\textbf{Sharpening Filter}**: A filter that enhances the edges and details in an image.
- **\textbf{Codewords}**: Visual features extracted from an image, often represented as small image patches or visual words.
- **\textbf{Edge Detection}**: A technique in image processing used to identify points in an image where there is a significant change in intensity.
- **\textbf{Edges}**: Curves in an image across which brightness changes significantly.
- **\textbf{Intensity Profile}**: A plot of intensity values along a horizontal or vertical scanline of an image.
- **\textbf{Corner Detection}**: A technique in image processing used to identify points in an image where there are significant changes in intensity in multiple directions.
- **\textbf{Corners/Junctions}**: Points in an image where edges meet or intersect.
- **\textbf{Harris Detector}**: A corner detection algorithm that identifies corners based on the changes in the image intensity when a small window is shifted.

- **\textbf{Vision Pyramids}**: A hierarchical representation of an image at multiple resolutions, often used for multi-scale feature extraction.
- **\textbf{Gaussian Pyramids}**: A type of vision pyramid where each level is created by blurring and subsampling the previous level using a Gaussian filter.
- **\textbf{Moving Average (2D)}**: A filtering technique that smooths an image by averaging the intensity values in a local neighborhood around each pixel.
- **\textbf{Image Derivatives}**: Approximations of the rate of change of intensity in an image, often used for edge detection.
- **\textbf{Canny Edge Detector}**: An edge detection algorithm that uses multiple steps, including gradient calculation, non-maximum suppression, and hysteresis thresholding, to identify edges in an image.
- **\textbf{Gaussian Filter}**: A filter that smooths an image by convolving it with a Gaussian function.
- **\textbf{Derivative of Gaussian Filter}**: A filter that detects edges by convolving an image with the derivative of a Gaussian function.
- **\textbf{Anisotropic Gaussian}**: A 2D Gaussian function with different standard deviations along the x and y axes.
- **\textbf{Isotropic Gaussian}**: A 2D Gaussian function with the same standard deviation along the x and y axes.
- **\textbf{Convolutional Neural Network (CNN) for Image/Video Recognition}**: A type of artificial neural network designed to process data with a grid-like topology, such as images. CNNs use convolutional layers to extract features from the input data and pooling layers to reduce the dimensionality of the data.
- **\textbf{ImageNet Top-5 Classification Accuracy}**: A metric used to evaluate the performance of image classification models on the ImageNet dataset. It measures the percentage of times the correct class is among the top 5 predicted classes.
- **\textbf{AlexNet}**: One of the first deep convolutional neural networks to achieve high accuracy in image classification, particularly on the ImageNet dataset. It was notable for its use of GPUs for faster training.
- **\textbf{Dense Neural Network}**: A type of artificial neural network where each neuron in one layer is connected to every neuron in the next layer. This creates a fully connected network.
- **\textbf{Convolutional Neural Network (ConvNet)}**: A type of artificial neural network that uses convolutional layers to extract features from input data, typically images. It processes data in three dimensions (width, height, depth).
- **\textbf{Activation Function}**: A non-linear function applied to the output of a neuron to introduce non-linearity into the network. Examples include sigmoid and ReLU.
- **\textbf{Convolution Layer}**: A layer in a convolutional neural network that applies a filter (kernel) to the input data to extract features. The filter slides across the input data, performing element-wise multiplication and summation.
- **\textbf{Pooling Layer}**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps, typically by applying a max or average pooling operation.
- **\textbf{Convolution Filter}**: A small matrix of weights used in a convolutional layer to extract features from the input data. It slides across the input data, performing element-wise multiplication and summation.

- **\textbf{Input Map}**: A matrix representing a portion of an image or a feature map from a previous layer in a convolutional neural network. It serves as the input to a convolutional layer.
- **\textbf{Output Map}**: A matrix representing the output of a convolutional layer. It contains the extracted features from the input map.
- **\textbf{Zero Padding}**: A technique used in convolutional neural networks to add zeros around the borders of the input data, maintaining the dimensions of the output feature maps.
- **\textbf{Multidimensional Convolution}**: A convolution operation applied to input data with multiple channels (e.g., RGB channels in an image). Each channel is processed independently by a corresponding filter.
- **\textbf{Local Receptive Field}**: The small region of the input data that a neuron in a convolutional layer is connected to. This leads to sparse connectivity.
- **\textbf{Sparse Connectivity}**: A characteristic of convolutional neural networks where each neuron is connected to only a few neurons in the previous layer, unlike fully connected networks.
- **\textbf{Parameter Sharing}**: A technique used in convolutional neural networks where the same set of weights (parameters) is used across all spatial locations in a feature map.
- **\textbf{Rectified Linear Unit (ReLU)}**: A piecewise linear activation function defined as $f(x) = \max(0, x)$. It is commonly used in neural networks due to its computational efficiency and ability to avoid the vanishing gradient problem.
- **\textbf{Max Pooling}**: A pooling operation that selects the maximum value within a defined region (e.g., 2x2) of a feature map.
- **\textbf{Average Pooling}**: A pooling operation that calculates the average value within a defined region (e.g., 2x2) of a feature map.
- **\textbf{Backpropagation}**: An algorithm used to train neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights accordingly.
- **\textbf{Convolutional Kernel Layer}**: A layer in a convolutional neural network that performs the convolution operation, applying filters to extract features from the input data.
- **\textbf{Dimension Reduction Layer}**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps, typically using pooling operations.
- **\textbf{Fully Connection Layer}**: A layer in a neural network where each neuron is connected to every neuron in the next layer.
- **\textbf{Learned Convolution Filters}**: The weights of the convolutional filters in a convolutional neural network that are learned during the training process. These filters detect specific features in the input data.
- **\textbf{MNIST Dataset}**: A large database of handwritten digits that is commonly used for training and testing in the field of machine learning.
- **\textbf{CIFAR-10 Dataset}**: A large dataset of 32x32 color images, commonly used for training and testing image classification models.
- **\textbf{LeNet-5}**: A convolutional neural network architecture specifically designed for handwritten digit recognition using the MNIST dataset.
- **Held Out Data**: A subset of the training data used to tune hyperparameters of a model, preventing overfitting to the training data.

- **Test Data:** A subset of the data used to evaluate the performance of a trained model on unseen data, providing an unbiased estimate of generalization ability.
- **Training Data:** The subset of data used to train a machine learning model, allowing the model to learn patterns and relationships from the data.
- **Hyperparameters:** Parameters that control the learning process of a model, such as learning rate, regularization strength, or number of layers in a neural network. They are not learned from the data but are set by the user.
- **Regularization:** A technique used to prevent overfitting by adding a penalty term to the loss function, discouraging overly complex models with large weights.
- **Regularization Coefficient (λ):** A hyperparameter that controls the strength of the regularization penalty in regularized regression. A larger λ leads to stronger regularization.
- **LASSO:** A type of regularization that uses the L1 norm of the weights as the penalty term in the loss function, leading to sparsity in the model's weights.
- **Learning Curves:** Graphs that plot the training and testing performance of a model as a function of the training set size or model complexity, providing insights into model bias and variance.
- **S-fold Cross Validation:** A model evaluation technique where the data is split into S folds, and each fold is used once as a validation set while the remaining folds are used for training. The average performance across all folds is used to estimate the model's generalization ability.
- **Data Analysis Pipeline:** A sequence of steps involved in analyzing data, including data collection, model design, parameter inference, and model application. Each step has potential sources of error that need to be considered.
- **Matrix Factorization:** A technique that decomposes a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Principal Component Analysis (PCA):** A linear dimensionality reduction technique that transforms data into a lower-dimensional space by identifying principal components, which are orthogonal directions that maximize the variance of the data.
- **Dimensionality Reduction:** The process of reducing the number of variables in a dataset while retaining as much important information as possible. This is often done to simplify data, improve visualization, or remove irrelevant features.
- **Low Rank Matrix Approximations:** Approximating a high-rank matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Embeddings:** A low-dimensional representation of data points, often obtained through dimensionality reduction techniques like matrix factorization. They project high-dimensional data into a lower-dimensional space while preserving relationships between data points.
- **Collaborative Filtering:** A recommender system technique that predicts user preferences based on the preferences of similar users.
- **Recommender Systems:** Systems that predict user preferences for items, such as movies, products, or news articles, based on past user behavior and other relevant data.
- **EigenSNPs:** Eigenvectors obtained from Principal Component Analysis (PCA) applied to Single Nucleotide Polymorphisms (SNPs) data. They represent directions of maximum variance in the genetic data, capturing patterns of genetic variation across populations.
- **Single Nucleotide Polymorphisms (SNPs):** Variations in a single nucleotide base (A, C, G, or T) in a DNA sequence that occur at a specific location in the genome and differ between individuals.

- **Baseline Predictor**: A component of a recommender system that provides a basic prediction of a user's rating for an item, based on overall average ratings and user/item biases.
- **Bias-Variance Tradeoff**: The trade-off between model complexity and prediction accuracy. A model that is too complex may overfit the training data, leading to high variance and poor generalization to new data. A model that is too simple may underfit the data, leading to high bias and poor accuracy.
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of a recommender system by calculating the square root of the average squared difference between predicted and actual ratings.
- **Netflix Prize**: A competition launched by Netflix in 2006 to improve the accuracy of its movie recommendation system by offering a *1millionprizeforasignificantimprovementinpredictionaccuracy*.
- **\textbf{Clustering}**: The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **\textbf{Supervised Learning}**: A machine learning paradigm where the algorithm learns from labeled data (input-output pairs) to predict outputs for new inputs.
- **\textbf{Unsupervised Learning}**: A machine learning paradigm where the algorithm learns from unlabeled data to discover patterns, structures, or relationships within the data.
- **\textbf{Euclidean Distance}**: The most commonly used distance measure for continuous data, calculated as $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$, where p is the number of attributes.
- **\textbf{Proximity}**: A general term indicating the similarity or dissimilarity between objects.
- **\textbf{Distance}**: A measure of dissimilarity between objects.
- **\textbf{Hamming Distance}**: A distance measure used for bit/binary vectors; it counts the number of bits that differ between two vectors.
- **\textbf{Matching Coefficient}**: A normalized Hamming distance, obtained by dividing the Hamming distance by the total number of attributes/bits.
- **\textbf{K-means Clustering}**: A popular clustering algorithm that partitions data into k disjoint subsets by iteratively assigning data points to the nearest cluster centroid and recalculating centroids until convergence.
- **\textbf{Inertia}**: In the context of K-means, the sum of squared distances from each point to its cluster centroid. Minimizing inertia is the objective of K-means.
- **\textbf{Elbow Method}**: A technique for choosing the optimal number of clusters (k) in K-means by plotting the inertia against different values of k and selecting the point where the decrease in inertia starts to level off (the "elbow").
- **\textbf{Purity}**: An evaluation metric for clustering that measures the extent to which clusters contain objects of a particular class. It assigns each cluster the class with the most numerous members and averages the maximum number of elements of any class within each cluster.
- **\textbf{Personas}**: Representations of different user types or target audiences, used to provide a common understanding across teams in product or service development.
- **\textbf{Principal Component Analysis (PCA)}**: A dimensionality reduction technique used to transform high-dimensional data into a lower-dimensional representation while preserving as much variance as possible.
- **\textbf{Intelligent Agent}**: A system that perceives its environment and takes actions that maximize its chances of success.

- **\textbf{Logic Reasoning}**: The process of using rules of deduction to derive a conclusion from given assumptions.
- **\textbf{Propositional Logic}**: The simplest form of logic, using propositional symbols and logical connectives to represent sentences and their relationships.
- **\textbf{Model (Propositional Logic)}**: An assignment of truth values (true/false) to each propositional symbol in a propositional logic formula.
- **\textbf{Logical Connectives}**: Symbols used in propositional logic to combine propositional symbols, including negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), implication ($\Rightarrow$), and biconditional ($\Leftrightarrow$).
- **\textbf{Tautology}**: A sentence that is true in all models.
- **\textbf{Satisfiable Sentence}**: A sentence that is true in at least one model.
- **\textbf{Unsatisfiable Sentence}**: A sentence that is true in no models.
- **\textbf{Literal}**: A propositional variable or its negation.
- **\textbf{Clause}**: A disjunction of literals.
- **\textbf{Conjunctive Normal Form (CNF)}**: A logical formula represented as a conjunction of clauses.
- **\textbf{Boolean Satisfiability Problem (SAT)}**: The problem of determining whether a given propositional formula is satisfiable.
- **\textbf{3-Satisfiability (3-SAT)}**: A restricted version of the SAT problem where each clause contains exactly three literals.
- **\textbf{Equisatisfiable}**: Two formulas are equisatisfiable if they have the same set of satisfying assignments.
- **\textbf{Model Checking}**: A technique for verifying the correctness of hardware or software systems by checking if they satisfy a given property.
- **\textbf{Bounded Model Checking (BMC)}**: A model checking technique that explores a finite number of steps in a system's execution.
- **\textbf{Safety Property}**: A property that states that a system should never reach a certain undesirable state.
- **\textbf{Liveness Property}**: A property that states that a system will eventually reach a certain desirable state.
- **\textbf{Classical Planning}**: The problem of finding a sequence of actions that achieves a given goal.
- **\textbf{Combinatorial Design}**: The study of mathematical structures with specific properties, often used in various applications.
- **Recursion**: A process where a function calls itself, eventually terminating in a base case.
- **Base Case**: The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case**: In a recursive approach, the same problem/method/data structure, but smaller, closer to the base case.
- **Breadth First**: A tree traversal method that visits nodes level by level.
- **Depth First**: A tree traversal method that traverses down a branch of a tree.

- **Binary Search Tree:** A tree data structure where each node has a value, its left child has a smaller value, and its right child has a larger value.
- **Search Tree:** A tree structure where each node represents a possible state in a search process.
- **Problem Solving as Search:** An approach to problem-solving that involves exploring possibilities to find a solution.
- **States:** Configurations of an environment in a search problem.
- **Actions:** Activities that move from one state to another in a search problem.
- **Search Algorithm:** Determines which actions should be tried in which order to find a solution in a search problem.
- **Successor Function:** A function that maps a state to its possible successor states.
- **World State:** Includes every detail of the environment in a search problem.
- **Search State:** Keeps only the details needed for planning (abstraction) in a search problem.
- **Rational Agent:** An agent whose actions maximize goal achievement, given available information.
- **Rational Behavior:** Doing the things which are expected to maximize goal achievement, given available information.
- **Satisfiability (SAT) Problem:** Determining whether there exists an assignment of truth values to the variables in a Boolean formula that makes the formula evaluate to true.
- **Conjunctive Normal Form (CNF):** A standardized way of representing Boolean formulas as a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals.
- **Literal:** A Boolean variable or its negation.
- **Clause:** A disjunction (OR) of literals.
- **Bounded Model Checking (BMC):** A technique for verifying temporal properties of a system by checking if a given model satisfies a property within a bounded number of steps.
- **Safety Property:** A property that states that a system will never reach an undesirable state.
- **Liveness Property:** A property that states that a system will eventually reach a desirable state.
- **Partial Assignment:** An assignment of truth values to a subset of the variables in a Boolean formula, leaving some variables unassigned.
- **Unit Clause:** A clause containing only one literal.
- **Unit Propagation (BCP):** A technique in SAT solving that efficiently assigns truth values to variables based on unit clauses.
- **Davis-Putnam-Logemann-Loveland (DPLL) Algorithm:** A recursive algorithm for solving the SAT problem that uses backtracking and Boolean constraint propagation.
- **Implication Graph:** A graph that represents the implications between variables in a SAT problem, used to visualize the search process of algorithms like DPLL.
- **Resolution Rule:** An inference rule in propositional logic that allows deriving a new clause from two clauses containing a literal and its negation.
- **Clause Learning:** A technique in SAT solving where clauses are learned from conflicts during the search process to avoid repeating similar mistakes.

- **Hamiltonian Path Problem:** Determining whether there exists a path in a graph that visits each vertex exactly once.
- **Graph Coloring Problem:** Assigning colors to the vertices of a graph such that no two adjacent vertices have the same color.
- **Cook's Theorem:** A fundamental theorem in computational complexity theory stating that the Boolean satisfiability problem is NP-complete.
- **Bias:** Systematic and repeatable errors in a data set or algorithm that can lead to unfair or discriminatory outcomes.
- **Fairness:** The property of a data set or algorithm that ensures equitable treatment of all individuals or groups, without discrimination.
- **Disparate Treatment:** A type of discrimination where decisions are (partly) based on a subject's protected attribute information.
- **Disparate Impact:** A type of discrimination where outcomes disproportionately hurt (or benefit) people with certain sensitive attribute values.
- **Proxies:** Seemingly neutral data points that can act as stand-ins for discriminatory factors, perpetuating bias in algorithms.
- **Anonymized Data:** Data that has had identifying information removed. However, this does not always guarantee privacy or prevent re-identification.
- **Interpretable Results:** Results from an AI algorithm that are easily understandable and explainable, increasing trust and transparency.
- **Group Fairness:** A fairness criterion that aims to equalize outcomes for different groups, ensuring that the probability of a particular outcome is similar across groups.
- **Individual Fairness:** A fairness criterion that aims to treat similar individuals similarly, regardless of group membership.
- **Automatic Prediction System:** A system that uses algorithms to make predictions automatically, often used in various applications like web advertising, lending, and ridesharing.

2 Problem Types

- **\textbf{Understanding the Nature of Intelligence:** This problem explores the fundamental question of what constitutes intelligence, both human and artificial. Different perspectives and definitions of AI are examined, including the Turing Test and its limitations.
- **\textbf{Overcoming the Limitations of Search Algorithms:** This problem addresses the computational challenges associated with exhaustive search in complex games like checkers and Go. The exponential growth of the search space necessitates the use of more efficient algorithms and machine learning techniques.
- **\textbf{Addressing the Challenges of Computer Vision:** This problem focuses on the difficulties involved in developing computer vision systems that can reliably interpret and understand images. The historical context of computer vision research is discussed, highlighting the gap between initial expectations and the ongoing challenges.
- **\textbf{Mitigating Bias and Ensuring Ethical AI Development:** This problem examines the ethical implications of AI systems, particularly the potential for bias and discrimination. The importance of responsible AI development and the need to address ethical concerns are emphasized.

- **\textbf{Developing Robust and Safe AI Systems}**: This problem explores the vulnerabilities of AI systems, such as their susceptibility to adversarial attacks. The need for robust and safe AI systems that can handle unexpected inputs and situations is highlighted.
- **\textbf{Designing Effective Game-Playing Algorithms}**: This problem focuses on the design and implementation of algorithms for playing games, including the use of search algorithms like Minimax and Alpha-Beta pruning. The challenges of representing game states, planning actions, and reasoning about opponents' moves are discussed.
- **\textbf{Bridging the Gap Between Different AI Domains}**: This problem explores the similarities and differences between seemingly disparate AI tasks, such as speech synthesis and game playing. The goal is to identify common underlying principles and approaches.
- **\textbf{Understanding the Components of the AI Stack}**: This problem examines the various components and subfields that constitute the AI stack, highlighting their interdependencies and relationships.
- **\textbf{Developing Autonomous Systems for Real-World Applications}**: This problem focuses on the challenges and opportunities associated with developing autonomous systems, such as self-driving cars. The reasons for selecting autonomous vehicles as an AI grand challenge are explored.
- **\textbf{Building Effective Conversational Agents}**: This problem explores the design and implementation of conversational agents, highlighting the challenges of representing knowledge, making decisions, and reasoning about the context of conversations.
- **\textbf{Applying AI to Real-World Problems}**: This problem examines the application of AI to real-world problems, such as fraud detection.
- **Representing Linear Transformations with Matrices**: Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic**: The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations using Matrices**: Systems of linear equations can be represented and solved efficiently using matrix operations, particularly using matrix inverses.
- **Matrix Transposition**: Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix, useful for data manipulation in AI.
- **Understanding Perceptrons**: A perceptron, a fundamental building block of neural networks, is introduced, illustrating its structure and function.
- **Parallelizing Matrix Multiplication**: Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Calculus in AI**: The slide highlights the importance of calculus, specifically derivatives, partial derivatives, and finding maxima/minima/gradients, in AI algorithms.
- **Calculating the Derivative of the Sigmoid Function**: The slide shows the step-by-step derivation of the derivative of the sigmoid function using calculus rules.
- **\textbf{Understanding Inductive Reasoning in Machine Learning}**: This problem explores how machine learning algorithms learn patterns from data examples to make predictions on new, unseen data. The example uses images of trains carrying or not carrying toxic chemicals to illustrate the concept.
- **\textbf{Image Recognition and Classification}**: This problem focuses on the challenges of teaching computers to classify images based on visual features. The example uses images of sheep, horses, and llamas to illustrate the difficulty of distinguishing between similar-looking animals.

- **\textbf{Building a Predictive Model from Data Examples}**: This problem addresses the process of constructing a machine learning model from a dataset, emphasizing the importance of representative data for accurate predictions.
- **\textbf{Linear Regression as a Machine Learning Problem}**: This problem introduces linear regression as a fundamental machine learning task, focusing on finding the best-fitting line through a set of data points. The example uses house prices and square footage to illustrate the concept.
- **\textbf{Multiple Linear Regression with Multiple Independent Variables}**: This problem extends linear regression to handle multiple independent variables, using the example of predicting CO2 emissions based on car weight and volume.
- **\textbf{Curve Fitting and Overfitting}**: This problem explores the trade-off between model complexity and generalization ability, demonstrating how overfitting can occur when a model is too complex for the data. The example uses polynomial curve fitting to illustrate the concept.
- **\textbf{Binary Classification using Linear Models}**: This problem investigates the use of linear models for binary classification tasks, focusing on the choice of loss function and optimization methods.
- **Image Classification Challenges**: This problem focuses on the difficulties in accurately classifying images due to variations in viewpoint, illumination, deformation, occlusion, and background clutter.
- **Representing Images Numerically**: This problem addresses the challenge of representing images as numerical data that can be processed by computers, highlighting the semantic gap between human perception and computer representation.
- **Finding Invariant Features**: This problem explores the need to identify features in images that remain consistent across different variations in viewpoint, lighting, and other factors, crucial for robust image classification.
- **Bag of Features Approach**: This problem focuses on using the Bag of Features method for image classification, which involves extracting local features, creating a visual vocabulary, and representing images as frequency distributions of visual words.
- **Noise Reduction in Images**: This problem addresses the challenge of removing noise from images, exploring techniques like mean filtering and moving averages to smooth out noisy pixel values.
- **Image Sharpening**: This problem focuses on enhancing image details by reversing the effects of blurring, using mathematical formulas and filters to increase the sharpness and clarity of images.
- **Document Representation for Retrieval**: This problem explores representing documents numerically using word frequencies for efficient information retrieval.
- **\textbf{Object Categorization using Bag of Features}**: This problem focuses on using the Bag of Features approach to categorize objects in images by representing them as histograms of visual features (codewords) and comparing them to category models in a model space.
- **\textbf{Edge Detection in Images}**: This problem involves identifying edges in images, which are locations of rapid changes in image intensity. The problem explores the use of image derivatives and filters to detect these edges, and the challenges posed by noise.
- **\textbf{Corner Detection in Images}**: This problem focuses on identifying corners in images, which are points where the image intensity changes significantly in multiple directions. The problem explores the use of the Harris detector and other methods to identify these corners.
- **\textbf{Noise Reduction in Images}**: This problem addresses the issue of noise in images, which can interfere with edge and corner detection. The problem explores the use of smoothing filters, such as Gaussian filters, to reduce noise before applying derivative operations.

- **\textbf{Multi-scale Image Representation using Vision Pyramids}**: This problem involves representing images at multiple scales (resolutions) using vision pyramids, such as Gaussian pyramids. This allows for feature extraction at different levels of detail.
- **\textbf{Object Detection and Recognition using Convolutional Neural Networks (CNNs)}**: CNNs are used to identify and locate objects within images and videos. Examples include facial landmark detection, car detection in highway scenes, and fruit recognition.
- **\textbf{Improving ImageNet Classification Accuracy over Time}**: This problem focuses on tracking the progress of image recognition models' accuracy on the ImageNet dataset over several years, highlighting the impact of advancements like AlexNet.
- **\textbf{Comparing Dense and Convolutional Neural Networks}**: This problem contrasts the architectures and processing methods of standard fully connected neural networks and convolutional neural networks, emphasizing the three-dimensional processing of ConvNets.
- **\textbf{Understanding the Function of an Artificial Neuron}**: This problem describes the mathematical operations performed by an artificial neuron, including weighted input summation and activation function application.
- **\textbf{Illustrating Convolution and Pooling Operations in CNNs}**: This problem demonstrates the mathematical calculations and visual effects of convolution and pooling layers in CNNs using numerical examples.
- **\textbf{Explaining the Advantages of Convolution over Fully Connected Networks}**: This problem explains why convolutional layers are preferred over fully connected layers in image processing, focusing on sparse connectivity and parameter sharing.
- **\textbf{Introducing the Rectified Linear Unit (ReLU) Activation Function}**: This problem defines and graphically represents the ReLU activation function, $f(x) = \max(0, x)$.
- **\textbf{Explaining Pooling Layers in CNNs}**: This problem describes the function of pooling layers in downsampling feature maps, illustrating max pooling and average pooling.
- **\textbf{Illustrating the Backpropagation Algorithm for Training Neural Networks}**: This problem visually represents the backpropagation process, showing the flow of information and error signals through the network layers.
- **\textbf{Presenting a Simple CNN Architecture}**: This problem visually depicts a basic CNN architecture, showing the sequence of convolutional, activation, pooling, and fully connected layers.
- **\textbf{Visualizing Learned Convolution Filters}**: This problem displays examples of learned convolution filters from a CNN, illustrating the types of features the network learns to detect.
- **\textbf{Illustrating Multidimensional Convolution}**: This problem visually demonstrates multi-dimensional convolution, showing how filters are applied to images with multiple color channels.
- **\textbf{Showing Biological Inspiration for Artificial Neural Networks}**: This problem uses diagrams of biological neurons and brain organization to illustrate the biological inspiration behind artificial neural networks.
- **\textbf{Introducing the MNIST Dataset of Handwritten Digits}**: This problem introduces the MNIST dataset, describing its size, content, and use in machine learning.
- **\textbf{Presenting the LeNet-5 Architecture for MNIST}**: This problem illustrates the LeNet-5 CNN architecture, detailing its layers and parameters for processing the MNIST dataset.
- **\textbf{Introducing the CIFAR-10 Dataset and State-of-the-Art Results}**: This problem introduces the CIFAR-10 dataset and presents a table showing the state-of-the-art accuracy achieved by various deep learning models.

- **Overfitting:** A model that performs well on training data but poorly on unseen data, indicating it has learned the noise in the training data rather than the underlying patterns.
- **Underfitting:** A model that performs poorly on both training and test data, indicating it is too simple to capture the underlying patterns in the data.
- **Diagnosing Overfitting:** Comparing the performance of a model on training and test data to identify overfitting, where a large discrepancy indicates overfitting.
- **Hyperparameter Tuning:** The process of adjusting hyperparameters (parameters that control the learning process, not learned from data) to optimize model performance. This often involves using a held-out validation set.
- **Controlling Model Complexity:** Techniques to prevent overfitting by limiting the model's capacity to fit noise in the training data. This can involve limiting polynomial degree, using regularization, or other methods.
- **Training/Test Cycle:** The process of splitting data into training, validation (held-out), and test sets to train, tune, and evaluate a model, respectively. The test set is never used during training or tuning.
- **S-fold Cross Validation:** A technique to evaluate model performance and tune hyperparameters by repeatedly splitting the data into training and validation sets, using each part as a validation set once.
- **Data Analysis Pipeline:** The complete process of collecting, designing, inferring parameters for, and applying a model to data, highlighting potential issues at each stage.
- **Learning Curves:** Graphs showing the relationship between model complexity and accuracy on training and test sets, used to diagnose underfitting and overfitting.
- **Bias-Variance Tradeoff:** The balance between model simplicity (low variance, high bias) and model complexity (high variance, low bias), aiming for a model that generalizes well to unseen data.
- **Matrix Factorization:** Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction:** Transforming data from a higher number of dimensions to a lower number of dimensions to discard irrelevant features, visualize high-dimensional data, or reflect the intrinsic dimensionality.
- **Low Rank Matrix Approximations:** Reducing the rank of a matrix to achieve dimensionality reduction, using equivalent definitions based on linearly independent subsets of columns/rows or matrix factorization.
- **Singular Value Decomposition (SVD):** Decomposing a matrix into three simpler matrices (U, S, V) to reveal latent factor affinities for entities and attributes.
- **Principal Component Analysis (PCA):** A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data.
- **Recommender System Algorithms:** Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Collaborative Filtering:** Predicting a user's rating for an item based on the ratings of similar users.
- **Modeling Systematic Biases:** Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Dealing with Missing Data in Recommender Systems:** Addressing the challenge of missing entries in the user-item rating matrix by replacing missing values or optimizing reconstruction error over known entries.

- **Clustering Data:** Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. Examples include clustering documents, images, genes, and movies.
- **Evaluating Clustering Results:** Assessing the "goodness" of resulting clusters, either subjectively (visual inspection of proximity matrices and PCA-reduced visualizations) or objectively (using known class labels and metrics like purity).
- **Choosing the Optimal Number of Clusters (k):** Determining the appropriate number of clusters (k) in K-means clustering by minimizing the sum of squared distances from each point to its cluster centroid (inertia) and identifying the "elbow" point in a plot of inertia versus k .
- **Understanding Distance Measures:** Using appropriate distance measures (Euclidean distance for continuous data, Hamming distance for discrete data) to quantify the similarity or dissimilarity between data points.
- **\textbf{Solving the Satisfiability Problem (SAT)}:** Determining whether a given propositional formula is satisfiable, meaning whether there exists an assignment of truth values to its variables that makes the formula true.
- **\textbf{Converting Logical Expressions to Conjunctive Normal Form (CNF)}:** Transforming arbitrary logical expressions into an equivalent CNF representation, which is a conjunction of clauses, each being a disjunction of literals.
- **\textbf{Applying SAT to Model Checking}:** Using SAT solvers to verify the correctness of hardware and software systems by encoding system properties and transitions into a CNF formula and checking its satisfiability.
- **\textbf{Applying SAT to Classical Planning}:** Utilizing SAT solvers to find optimal plans by encoding the planning problem's constraints and goals into a CNF formula and searching for satisfying assignments.
- **\textbf{Applying SAT to Combinatorial Design}:** Employing SAT solvers to find complex mathematical structures with specific properties, such as Latin squares or Steiner systems, by encoding the constraints of the design into a CNF formula.
- **\textbf{Reducing SAT to 3-SAT}:** Transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals, by introducing new variables and creating additional clauses.
- **Representing Recursion:** This problem focuses on understanding and applying the concept of recursion in AI, including its use in methods and data structures like linked lists and trees. Examples include the Fibonacci sequence and tree traversal.
- **Problem Solving as Search:** This problem explores the representation of problems as search spaces, defining states, actions, and search algorithms to find solutions. Examples include mazes, the 8-puzzle, and pathfinding on a map.
- **Optimizing Recursive Algorithms:** This problem addresses the efficiency of recursive algorithms by identifying and removing redundant calculations. The Fibonacci sequence is used as an example to illustrate the optimization process.
- **Representing Problems as Graphs:** This problem focuses on representing problems using graph structures, considering multiple paths to a solution and the implications for search algorithms. Examples include the 8-puzzle and pathfinding problems.
- **Defining State Spaces:** This problem involves defining appropriate state spaces for AI problems, distinguishing between world states (containing all details) and search states (containing only relevant details for planning). The Pac-Man game is used as an example.

- **\textbf{Solving the Satisfiability Problem (SAT)}**: This problem focuses on determining whether a given Boolean formula can be satisfied by assigning truth values to its variables. Several methods for solving SAT problems are discussed, including the DPLL algorithm and the use of unit propagation.
- **\textbf{Overcoming Exponential Complexity Growth in Automated Reasoning}**: This problem addresses the challenge of exponential complexity growth in various domains, such as VLSI verification, military logistics, chess, and deep space mission control. The lecture explores how advances in SAT solving have mitigated this challenge.
- **\textbf{Encoding Real-World Problems into SAT}**: This problem involves translating real-world problems, such as the Hamiltonian Path Problem and graph coloring, into a format solvable by SAT solvers. The lecture demonstrates how to encode these problems into CNF formulas.
- **\textbf{Applying SAT Solvers to Real-World Problems}**: This problem focuses on the practical application of SAT solvers to diverse domains, including model checking (hardware/software verification), classical planning, and combinatorial design.
- **Identifying and Mitigating Bias in Data and Algorithms**: This problem focuses on identifying and addressing biases present in datasets and the resulting algorithms, which can lead to unfair or discriminatory outcomes. Examples include bias in hiring algorithms, loan applications, and criminal justice risk assessments.
- **Ensuring Data Privacy and Security**: This problem addresses the challenges of protecting sensitive personal data used in AI systems, while also ensuring the security of these systems from malicious attacks. Examples include anonymization techniques and the trade-offs between privacy and utility.
- **Balancing Accuracy and Interpretability in AI Models**: This problem explores the trade-off between the accuracy of AI models and their interpretability. Highly accurate models may be difficult to understand, making it hard to trust their outputs or identify potential biases. Examples include the choice between neural networks and logistic regression in medical diagnosis.
- **Defining and Achieving Fairness in AI**: This problem focuses on defining and operationalizing fairness in AI systems, considering different notions of fairness (e.g., group fairness, individual fairness) and their potential conflicts. Examples include the challenges of ensuring fairness in predictive policing and recidivism prediction.

3 Algorithm Solutions

- **Minimax Search**: A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning**: An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Matrix Addition**: $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication**: $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication**: $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Inverse**: $AA^{-1} = A^{-1}A = I$
- **Solving Linear Equations using Matrix Inverses**: $Ax = b \Rightarrow x = A^{-1}b$
- **Derivative of Sigmoid Function**: $\frac{d}{dx} \sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$

- **Multiple Linear Regression:** Find the optimal coefficients β that minimize the sum of squared errors between the observed and predicted values. This is achieved by solving the equation $\beta = (X^T X)^{-1} X^T y$, where y is the vector of observed values, X is the design matrix of independent variables, and β is the vector of coefficients.
- **Perceptron Algorithm:** Iteratively update the weight vector w by adding $y_n x_n$ if the data point x_n is misclassified (i.e., $\text{sign}(w^T x_n) \neq y_n$).
- **Bag of Features:** Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier:** Assign label of nearest training data point to each test data point
- **Moving Average Filter:** Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average Filter:** Add weights to the moving average to give different importance to neighboring pixels
- **Image Sharpening:** $f_{\text{sharp}} = f + \alpha(f - f_{\text{blur}})$ where f is the original image, f_{blur} is the blurred image, and α is a scaling factor
- **Finite Difference Method:** $\frac{df}{dx}[x, y] \approx F[x + 1, y] - F[x, y]$
- **Image Gradient Magnitude:** $\|\nabla f\| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction:** $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Sum of Squared Differences (SSD) Error:** $E(u, v) = \sum_{(x, y) \in W} [I(x + u, y + v) - I(x, y)]^2$
- **Anisotropic Gaussian:** $G_{\sigma_x, \sigma_y}(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2}) + (\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian:** $G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{r^2}{2\sigma^2}}$
- **Convolutional Neural Network (CNN):** A neural network architecture designed for processing grid-like data, such as images, using convolutional layers to extract features and pooling layers to reduce dimensionality.
- **ReLU Activation Function:** A piecewise linear activation function defined as $f(x) = \max(0, x)$, introducing non-linearity into the network.
- **Backpropagation Algorithm:** An algorithm used to train neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights iteratively to minimize the loss.
- **Max Pooling:** A pooling operation that selects the maximum value within a defined region (e.g., 2x2) of a feature map, reducing the spatial dimensions while preserving the depth.
- **Average Pooling:** A pooling operation that calculates the average value within a defined region (e.g., 2x2) of a feature map, reducing the spatial dimensions while preserving the depth.
- **LeNet-5 Architecture:** A specific CNN architecture designed for the MNIST dataset, consisting of convolutional layers, pooling layers, and fully connected layers.
- **Regularized Linear Regression:** $E(w) = \text{RSS}(w) + \lambda R(w)$, where $R(w)$ is either $\|w\|_2^2$ or $\|w\|_1$, and λ controls the trade-off between training error and complexity.
- **S-fold Cross Validation:** Split the training data into S equal parts; use each part as a validation dataset and the others as a training dataset; choose the hyperparameter leading to the best average performance.

- **Singular Value Decomposition (SVD)**: $A = USV^T$, where A is the input matrix, U and V are orthogonal matrices, and S is a diagonal matrix containing singular values.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data. The eigenvectors of the covariance matrix correspond to the principal components.
- **Matrix Factorization for Recommender Systems**: Approximates a user-item rating matrix as a product of two lower-dimensional matrices representing user and item latent factors. $r_{i,j} \approx u_i \cdot v_j$, where $r_{i,j}$ is the rating of user i for item j , u_i is the user's latent factor vector, and v_j is the item's latent factor vector.
- **Modeling Systematic Biases in Recommender Systems**: $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where r_{ij} is the rating, μ is the overall mean rating, b_i is the user bias, b_j is the item bias, and $u_i \cdot v_j$ represents user-item interactions.
- **Root Mean Square Error (RMSE)**: $\frac{1}{|R|} \sum_{(u,i) \in R} (r_{ui} - \hat{r}_u)^2$, where R is the set of ratings, r_{ui} is the actual rating, and $\hat{r}_u$ is the predicted rating.
- **K-means Clustering**: The algorithm iteratively assigns data points to the nearest cluster centroid, then recomputes the centroids as the mean of the points in each cluster, repeating until convergence.
- **Euclidean Distance**: $d_E(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$, where x and y are p -dimensional vectors.
- **Hamming Distance**: Counts the number of differing bits between two binary vectors.
- **Matching Coefficient**: Hamming distance normalized by the number of attributes/bits.
- **Purity**: $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^K \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of points of class j in cluster i .
- **DPLL Algorithm**: A backtracking-based algorithm for solving the Boolean satisfiability problem (SAT). It systematically assigns truth values to variables, checking for contradictions and backtracking when necessary.
- **Conversion to CNF**: A set of rules to transform any logical expression into an equivalent Conjunctive Normal Form (CNF) expression. This is crucial for using SAT solvers, as many solvers are designed to work specifically with CNF.
- **Recursive Fibonacci**: $\text{fib}(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{otherwise} \end{cases}$
- **Recursive Binary Search Tree Search**: Recursively check the current node's value (V) against the target value (X). If $V == X$, return 'True'. If $X < V$, recursively search the left subtree; if $X > V$, recursively search the right subtree. If no children exist, return 'False'.
- **Breadth-First Search**: Explore nodes level by level, adding children to a queue.
- **Depth-First Search**: Explore a branch completely before moving to another branch.
- **Optimized Fibonacci (Dynamic Programming)**: Store previously calculated Fibonacci numbers in a cache to avoid redundant computations.
- **Naïve SAT**: A recursive algorithm that explores all possible variable assignments to determine the satisfiability of a propositional formula. It checks for immediate satisfiability or unsatisfiability, then recursively calls itself with assignments of true and false to an unassigned variable, backtracking if both fail.
- **DPLL**: A recursive algorithm that improves upon the Naïve SAT algorithm by incorporating Boolean Constraint Propagation (BCP) to efficiently identify and resolve unit clauses, signifying a sub-algorithm within DPLL that repeatedly identifies and assigns values to literals in unit clauses (clauses with only one unassigned literal).

- **Resolution Rule:** An inference rule that derives a new clause from two existing clauses containing a literal and its negation. The new clause combines the remaining literals from the original clauses using logical OR.
- **Clause Learning:** A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a concise reason (a clause) is learned to prevent similar conflicts in the future, improving efficiency.
- **Hamiltonian Path Encoding:** A method to encode the Hamiltonian Path Problem into a SAT problem. It uses Boolean variables h_{ij} representing whether the i -th vertex visited is vertex j . Constraints are added to ensure the path uses only existing edges and visits each vertex exactly once.
- **Graph Coloring Encoding:** A method to encode the Graph Coloring Problem into a SAT problem. It uses Boolean variables to represent the assignment of colors to vertices, with constraints ensuring that adjacent vertices have different colors.
- **Fairness through Blindness:** Ignore protected attributes during model training and prediction.
- **Group Fairness:** $P(\text{outcome}|S) = P(\text{outcome}|T)$, where S and T are two groups. This is insufficient as a fairness metric.
- **Individual Fairness:** Treat similar individuals similarly; similar individuals should have similar outcomes. How to achieve this is an open research question.
- **Detecting and Removing Bias from Models:** Keep bias features in the data during training, then remove what was learned from these features after training.